3.1 Пакеты и модули

Организация кода, границы доступа и управление зависимостями.

- Пакет объединяет исходные файлы и задаёт пространство имён.
- Модуль объединяет пакеты, версии и зависимости.
- Практика: создадим модуль с пакетом calc и используем его из main.

Пакет и модуль: разные уровни

- Исходные файлы обычного пакета находятся в одной директории.
- В файлах пакета указывают одно имя в объявлении package. Исключение для внешних тестов: имя с суффиксом _test.
- Корень модуля определяется файлом go.mod; внутри модуля может быть несколько пакетов.
- Пакет main с функцией main создаёт запускаемую программу. Другие пакеты можно использовать как библиотеки.

Первый запуск

main.go

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, Go!")
}
```

Команды в директории проекта

```
go mod init example.com/mathops
go run .
go build .
```

go run . собирает и запускает пакет текущей директории; go build . создаёт исполняемый файл для пакета main.

Несколько файлов одного пакета

Два отдельных файла в одной директории

```
// main.go
package main
func main() { println(add(2, 3)) }

// add.go (отдельный файл)
package main
func add(a, b int) int { return a + b }
```

- Запуск: go run . или go run main.go add.go.
- При go run main.go второй файл не включается в список файлов: add не будет найдена.

Структура собственного модуля

Дерево проекта

```
mathops/
  go.mod           // module example.com/mathops
  main.go          // package main
  calc/
    add.go         // package calc
  internal/
    settings/
      config.go    // package settings
```

- Путь импорта calc: example.com/mathops/calc.
- Имя пакета обычно совпадает с последней частью пути, но это соглашение, а не обязательное равенство.
- Относительный импорт ./calc в модульном проекте не используем.

Экспорт и использование пакета

Файлы проекта example.com/mathops

```
// calc/add.go
package calc
func Add(a, b int) int { return a + b }
func normalize(n int) int { return n }

// main.go (отдельный файл)
package main
import (
    "fmt"
    "example.com/mathops/calc"
)
func main() { fmt.Println(calc.Add(2, 3)) }
```

Add начинается с заглавной буквы и экспортируется. normalize доступна только внутри пакета calc.

Стандартные, внешние и собственные пакеты

- Стандартная библиотека поставляется с Go: fmt, math, context, net/http.
- Внешние пакеты приходят из других модулей. Подключаем нужную версию через go get.
- Собственные пакеты лежат внутри вашего модуля и импортируются по его пути.
- Встроенные функции len, make, append и тип error не требуют импорта. Это элементы языка, а не отдельный пакет.

Импорты: обычный и с псевдонимом

Фрагмент: последние две строки внутри функции

```
import (
    "fmt"
    stdmath "math"
    "example.com/mathops/calc"
)

fmt.Println(stdmath.Pi)
fmt.Println(calc.Add(2, 3))
```

- Псевдоним помогает различать одинаковые имена или уточнить назначение пакета.
- Неиспользуемый обычный импорт вызывает ошибку компиляции.
- Зависимости пакетов не должны образовывать циклы.

Импорт для инициализации и импорт через точку

Независимые примеры объявления import

```
import _ "example.com/dbdriver"

// Пример альтернативного объявления импорта:
import . "math"
```

- Импорт с _ выполняет инициализацию пакета, не вводя имя для обращений к нему. Так могут регистрироваться драйверы.
- Импорт с точкой позволяет писать Pi вместо math.Pi, но делает происхождение имён менее очевидным.
- В обычном коде предпочтителен явный импорт с именем пакета.

Модули и версии зависимостей

Пример команд; версия указана явно

```
go get github.com/samber/lo@v1.47.0
go mod tidy
go list -m all
```

- go.mod задаёт путь модуля, директиву go и требования к зависимостям.
- go.sum содержит контрольные суммы загруженных модулей; это не список всех импортов.
- go mod tidy согласует зависимости с исходниками и тестами.
- Пометка indirect относится к зависимости, а не к способу записи import.

Кэш модулей и vendor

Команды в корне модуля

```
go env GOMODCACHE
go mod vendor
go build -mod=vendor .
```

- Кэш модулей хранит скачанные версии зависимостей. Типичный путь: GOPATH/pkg/mod.
- go mod vendor копирует необходимые пакеты зависимостей в vendor и создаёт modules.txt.
- После изменения зависимостей обновите vendor повторно. Не редактируйте его как собственный исходный код.

Граница internal

Пример структуры и импорта

```
example.com/mathops/
  internal/settings/
  cmd/tool/

// В cmd/tool допустим импорт:
import "example.com/mathops/internal/settings"
```

- Код ниже родительского каталога internal может импортировать его пакеты.
- Пакет из другой ветки дерева не получает доступ только потому, что знает путь.
- Используйте internal для деталей реализации, которые не должны становиться публичным API.

Публикация библиотеки

- Создайте библиотечный пакет и задайте в go.mod путь репозитория, например example.com/team/hello.
- Опубликуйте исходники в доступном системе Go репозитории и создайте тег версии.
- В проекте-потребителе добавьте зависимость: go get example.com/team/hello@v1.0.0.
- Импортируйте пакет по его пути; после изменения импортов выполните go mod tidy.
- Для мажорных версий v2 и выше учитывайте суффикс версии в пути модуля.

Что должно получиться

- Модуль example.com/mathops содержит пакеты main и calc.
- go run . печатает результат вызова calc.Add.
- Внешний пакет видит экспортируемые имена, но не normalize.
- Вы различаете import, go get, go mod tidy, кэш зависимостей и vendor.

Материалы и источники

Учебная реконструкция по содержанию урока «Пакеты и модули» курса Go и найденным материалам. Это новая подготовленная версия, не утраченный оригинальный PDF.

- Официальная документация: модули и зависимости
- Спецификация: пакеты, импорты и экспорт имён
- Справка команды go: build, run, mod и internal

https://go.dev/ref/mod
https://go.dev/ref/spec#Packages
https://pkg.go.dev/cmd/go